

Об'єктно-орієнтоване програмування

Частина 3: наслідування, композиція

Наслідування

Механізм наслідування забезпечує можливість зміни функціональності програм не шляхом зміни існуючого програмного коду, а за рахунок створення нових програмних компонентів, підкласів, які успадковують атрибути і методи батьківських класів і можуть модифікуватися та розширюватися. При цьому зміни у підкласах не зачіпають батьківських класів.

Для наслідування атрибутів іншого класу досить вказати ім'я цього класу в круглих дужках у заголовку інструкції `class`. Клас, який наслідує, називається **підкласом** (або **дочірнім класом**), а клас, від якого наслідуються атрибути і методи — **суперкласом** (або **батьківським класом**).

Екземплярам підкласу стають доступні атрибути і методи як суперкласу, так і підкласу. Якщо ці атрибути чи методи однойменні, то перевага надається тим, які належать підкласу. Це чимось схоже на області видимості функцій — якщо функція не знаходить імені всередині себе, вона починає шукати в охоплюючій її функції чи програмі. У випадку ООП пошук імен здійснюється по ієрархічному дереву класів аж до його верхівки.

Для демонстрації механізму наслідування створимо на основі класу `Person` (описаний у частині 2) клас `Employee`, який репрезентує працівника. Крім атрибутів `name` (ім'я) та `age` (вік), він матиме характерний для працівників атрибут `occupation` (посада):

```
class Employee(Person):

    counter = 0

    def __init__(self, name, age, occupation):
        super().__init__(name, age)
        self.occupation = occupation
        Employee.counter += 1

    def __str__(self):
        return f"{self.occupation.capitalize()} - {super().__str__()}"
```

Щоб показати принцип застосування атрибутів класу, спільних для всіх екземплярів, у класі `Employee` передбачено лічильник екземплярів `counter`. У конструкторі підкласу `Employee` викликано конструктор його суперкласу `Person`, щоб створити атрибути `name` і `age`, а потім створено додатковий атрибут `occupation` та виконано збільшення лічильника екземплярів на одиницю (так як конструктор автоматично запускається під час інстанціації екземпляра).

Аналогічно у методі `__str__()` використовується результат однойменного методу суперкласу, хоча можна було би сформувати рядок для відображення заново чи по-іншому.

Створимо та відобразимо декількох працівників (екземплярів класу `Employee`):

```
employee_1 = Employee('Alice', 20, 'manager')
employee_2 = Employee('Bob', 25, 'operator')
print(employee_1)    # Manager - Alice, 20 years old.
print(employee_2)    # Operator - Bob, 25 years old.
```

Переконаємося, що унаслідувався описаний в класі `Person` метод `grew_up()`, тобто, що він доступний з екземплярів класу `Employee`:

```
employee_1.grew_up()
print(employee_1)    # Manager - Alice, 21 years old.
```

Лічильник екземплярів також функціонує:

```
print(Employee.counter)    # 2
```

Якщо функціональність деякого методу суперкласу не задовільняє потреб підкласу, можна виконати заміщення методів, тобто написати у підкласі новий метод з таким самим іменем. При цьому доступ до однойменного методу суперкласу не втратиться, його можна буде викликати так само, як у конструкторі класу `Employee` викликався конструктор його суперкласу `Person`.

Також у підклас можна додавати яку завгодно кількість нових методів. Ці методи будуть доступні для підкласу, але не для суперкласу. Наприклад, додамо метод переведення на іншу посаду:

```
class Employee(Person):
    ...
```

```
def transfer(self, new_occupation):
    self.occupation = new_occupation

employee_2.transfer('boss')
print(employee_2)    # Boss - Bob, 25 years old.
```

Python також підтримує множинне успадкування, тобто успадкування від кількох класів, загальний синтаксис якого

```
class Імя_класу(Суперклас_1, Суперклас_2, ...):
    # тіло класу
```

Композиція

Композиція в ООП — спосіб відображення взаємовідношень між об'єктами, який полягає у вбудовуванні в клас-контейнер об'єктів інших класів.

Клас Employee має обмеження. Він репрезентує тільки одного працівника. Щоб мати можливість репрезентувати персонал деякої організації, створимо клас Staff, який використовуватиме композицію, тобто міститиме в собі список працівників.

```
class Staff:
    def __init__(self, staff=None):
        if staff==None:
            self.staff = []
        else:
            self.staff = staff

    def __add__(self, employee):
        self.staff.append(employee)

    def __str__(self):
        s = 'Staff:\n'
        for employee in self.staff:
            s += str(employee) + '\n'
        return s

    def sort(self):
        self.staff.sort(key = lambda x: x.age)

    def transfer(self, name, new_occupation):
        for employee in self.staff:
```

```
if employee.name == name:
    employee.transfer(new_occupation)
    break
```

У конструкторі класу Staff вказано, що інстанціація екземпляра здійснюється на основі списку (у нашому випадку списку працівників). Значення за замовчуванням None забезпечує створення порожнього списку у конструкторі класу, якщо конкретного списку не буде передано. При цьому [] не можна використовувати в якості значення за замовчуванням!!!

Для додавання працівника перевантажено оператор "+" (метод __add__()).

Також описано методи sort() і transfer(). Перший з них упорядковує список працівників за віком, а другий переводить певного працівника на іншу посаду.

Метод __str__() формує рядок, який відображає інформацію про персонал у зручному для сприйняття вигляді.

Продемонструємо роботу з екземпляром класу Staff:

```
employee_1 = Employee('Alice', 20, 'manager')
employee_2 = Employee('Bob', 25, 'operator')
employee_3 = Employee('Mary', 15, 'manager')

staff = Staff()

for item in [employee_1, employee_2, employee_3]:
    staff + item

print(staff)
# Staff:
# Manager - Alice, 20 years old.
# Operator - Bob, 25 years old.
# Manager - Mary, 15 years old.

staff.sort()
print(staff)
# Staff:
# Manager - Mary, 15 years old.
# Manager - Alice, 20 years old.
# Operator - Bob, 25 years old.

staff.transfer('Mary', 'boss')
```

```
print(staff)
# Staff:
# Boss - Mary, 15 years old.
# Manager - Alice, 20 years old.
# Operator - Bob, 25 years old.
```